Controllers

data layer between the view and model

screenshots from:
http://guides.rubyonrails.org/action_controller_overview.html

naming convention

something plural with "Controller" on the end

```
RoomsController
```

creating a controller

subclass ApplicationController

each public method inside the class is an "action"

```
$ rails generate controller NAME [action action]
```

Try Me

create a zebra controller

action

the destination of a route

(also, a method in a controller)

Try Me

add an index action to the zebra controller

add an index view

run the server and try navigating to localhost:3000/zebras

default views

<your action>.<format>.erb

Try Me

for now, just print the total number of zebras on it.

params

hash available in every action
holds query string parameters and post data

Try Me

inspect params from the index action

query string

data after the ? in a url

Try Me

print out (in the view) each key value pair in the query string. put a <br> between each pair

arrays in params

```
?ids[]=1&ids[]=2&ids[]=3
```

will become

```
params[:ids]
```

hash in params

To send a hash you include the key name inside the brackets:

```
<form accept-charset="UTF-8" action="/clients" method="post">
  <input type="text" name="client[name]" value="Acme" />
  <input type="text" name="client[phone]" value="12345" />
  <input type="text" name="client[address][postcode]"
value="12345" />
  <input type="text" name="client[address][city]" value="Carrot
City" />
</form>
```

respond_to

requests for non-html data formats

```
class UsersController < ApplicationController
  def index
    @users = User.all
    respond_to do |format|
      format.html # index.html.erb
      format.xml  { render xml: @users}
      format.json { render json: @users}
    end
  end
end
```

filter

non-action (private) methods in a controller that you configure to run before, after or around an action

before_action

```
class ApplicationController < ActionController::Base
  before_action :require_login

  private

  def require_login
    unless logged_in?
      flash[:error] = "You must be logged in to access this
section"
      redirect_to new_login_url # halts request cycle
    end
  end
end
```

after_action

has access to the response data

around_action

```
class ChangesController < ApplicationController
  around_action :wrap_in_transaction, only: :show

  private

  def wrap_in_transaction
    ActiveRecord::Base.transaction do
      begin
        yield
      ensure
        raise ActiveRecord::Rollback
      end
    end
  end
end
```

Routing

maps urls to controller actions
(in config/routes.rb)

screenshots from:
http://guides.rubyonrails.org/routing.html

many to one. first match wins.

Routing Parameters

also available in params inside an action

```
get '/clients/:status' => 'clients#index', foo: 'bar'
```

Path Generation

```
get '/patients/:id', to: 'patients#show', as: 'patient'
```

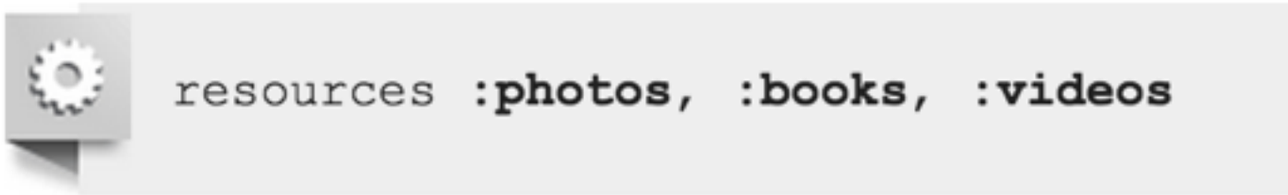
and your application contains this code in the controller:

```
@patient = Patient.find(17)
```

and this in the corresponding view:

```
<%= link_to 'Patient Record', patient_path(@patient) %>
```

resources



restrictions

```
resources :photos, only: [:index, :show]
```

name-spacing

```
namespace :admin do
  resources :articles, :comments
end
```

This will create a number of routes for each of the `articles` and `comments` controller. For `Admin::ArticlesController`, Rails will create:

HTTP Verb	Path	Controller#Action	Named Helper
GET	/admin/articles	admin/articles#index	admin_articles_path
GET	/admin/articles/new	admin/articles#new	new_admin_article_path
POST	/admin/articles	admin/articles#create	admin_articles_path

urls from objects

```
<%= link_to 'Ad details', magazine_ad_path(@magazine, @ad) %>
```

```
<%= link_to 'Edit Ad', [:edit, @magazine, @ad] %>
```

static segments

```
get ':controller/:action/:id/with_user/:user_id'
```

query string

```
get ':controller/:action/:id'
```

An incoming path of `/photos/show/1?user_id=2`

defaults

```
get 'photos/:id', to: 'photos#show', defaults: { format: 'jpg' }
```

naming routes

```
get ':username', to: 'users#show', as: :user
```

redirect

```
get '/stories', to: redirect('/articles')
```

Rendering

render

send a response
called from actions

screenshots from:
http://guides.rubyonrails.org/layouts_and_rendering.html

nothing

```
render nothing: true
```

not actually nothing, because a response is sent (headers but no data)

switching views

```
def update
  @book = Book.find(params[:id])
  if @book.update(book_params)
    redirect_to(@book)
  else
    render "edit"
  end
end
```

switching controllers

```
render "products/show"
```

render from string

```
render plain: "OK"
```

```
render html: "<strong>Not Found</strong>".html_safe
```

render from an object

```
render json: @product
```

```
render xml: @product
```

Try Me

create a route that returns all the zebras in json format
(without using a view)

layouts

wrapper views. use yield to render the inner view

```
<html>
  <head>
  </head>
  <body>
  <%= yield %>
  </body>
</html>
```

:layout

```
render layout: "special_layout"
```

```
render layout: false
```

content_for

yielding multiple sections

```
<html>
  <head>
  <%= yield :head %>
  </head>
  <body>
  <%= yield %>
  </body>
</html>
```

```
<% content_for :head do %>
  <title>A simple page</title>
<% end %>

<p>Hello, Rails!</p>
```

```
<html>
  <head>
  <title>A simple page</title>
  </head>
  <body>
  <p>Hello, Rails!</p>
  </body>
</html>
```

Readings

sections 1-8 of http://guides.rubyonrails.org/action_controller_overview.html

sections 1-3.8 of http://guides.rubyonrails.org/routing.html